

Proyecto Integrador Módulo 3

TechCore, Análisis de Ventas

Avance 3, Dashboard Interactivo en Power BI

Simón Bedoya

Soy Henry, Carrera de Data Science

2026

Descripción del Avance

Completadas las etapas de limpieza de datos y modelado relacional, DataVision Analytics encomienda la tercera y última fase del proyecto para TechCore. El propósito de este avance es importar el modelo relacional generado en Python dentro de Power BI y desarrollar un dashboard interactivo que transforme los datos en información estratégica para la toma de decisiones.

El trabajo abarca la construcción del modelo de datos dentro de Power BI, la creación de medidas de cálculo con el lenguaje DAX, la definición de jerarquías de navegación, la implementación de segmentaciones dinámicas y el diseño de las visualizaciones que comunican los resultados del análisis. El producto final es un panel de control que permite a la dirección de TechCore explorar su desempeño comercial de forma interactiva.

Objetivo del Avance

El objetivo es construir un dashboard que responda a las preguntas clave del negocio: cuánto se vende, dónde, qué marcas y productos lideran, cómo evoluciona el desempeño en el tiempo, y quién compra. El panel debe permitir filtrar la información por ciudad, marca, método de pago, período y edad del cliente, y debe habilitar la navegación desde una vista general hacia el detalle mediante jerarquías.

Importación del Modelo Relacional

El punto de partida es el archivo modeloVentas, generado por el notebook del Avance 2, que contiene las siete tablas del modelo relacional en hojas separadas. Al importarlo en Power BI, cada hoja se convierte en una tabla independiente dentro del modelo.

Power BI intentó detectar de forma automática las relaciones entre las tablas, guiándose por la coincidencia de nombres en las columnas de identificador. La detección resultó correcta en las seis relaciones, lo que confirma que el diseño de nombres del modelo fue coherente. Aun así, cada relación se verificó de forma individual, revisando su cardinalidad y la dirección del filtro cruzado.

Relación	Cardinalidad	Filtro
Ciudades a Sucursales	Uno a varios	Único
Sucursales a Facturas	Uno a varios	Único
Clientes a Facturas	Uno a varios	Único
Vendedores a Facturas	Uno a varios	Único
Facturas a DetalleFacturas	Uno a varios	Único
Productos a DetalleFacturas	Uno a varios	Único

La dirección del filtro se mantuvo en modo único en todas las relaciones, lo que significa que el filtro fluye desde la tabla del lado uno hacia la del lado varios. El filtro bidireccional, aunque disponible, puede producir resultados inesperados en modelos con varias tablas relacionadas, y no era necesario en este caso.

Configuración de los identificadores

Las columnas de identificador, como FacturaID o ProductoID, se configuraron con el resumen desactivado. Power BI, al detectar columnas numéricas, asume por defecto que puede sumarlas. Sumar identificadores no tiene sentido, porque un identificador no es una cantidad sino

una etiqueta. La misma configuración se aplicó a las columnas de año y mes, que son referencias temporales y no valores acumulables.

La Tabla de Calendario

El modelo original trae la fecha dentro de la tabla de Facturas, pero Power BI trabaja mejor con una tabla de calendario dedicada. Esta tabla contiene una fila por cada día del período analizado, con sus atributos temporales desglosados, y es la que habilita el correcto funcionamiento de las jerarquías de fecha y las funciones de inteligencia de tiempo.

La tabla se creó con la siguiente expresión.

```
Calendario =  
ADDCOLUMNS(  
    CALENDAR(  
        MIN(Facturas[Fecha]),  
        MAX(Facturas[Fecha])  
    ),  
    "Año", YEAR([Date]),  
    "Mes", MONTH([Date]),  
    "NombreMes", FORMAT([Date], "MMMM"),  
    "Dia", DAY([Date]),  
    "AñoMes", FORMAT([Date], "YYYY-MM")  
)
```

La función CALENDAR genera una secuencia continua de fechas entre la primera y la última venta registrada. La función ADDCOLUMNS le agrega las columnas derivadas de año,

mes, nombre del mes y día. La tabla resultante se relacionó con la de Facturas y se marcó de forma explícita como tabla de fechas, lo que le indica a Power BI que esa es la referencia temporal oficial del modelo.

Las Medidas DAX

DAX es el lenguaje de fórmulas de Power BI. Su característica fundamental es que las medidas se recalculan según el contexto: cuando el usuario filtra por una ciudad, la medida se vuelve a calcular considerando solo los datos de esa ciudad. Esta es la diferencia esencial frente a una columna calculada, que se computa una sola vez y queda fija.

Se crearon nueve medidas, que se explican a continuación.

VentasTotales

Es la medida base del dashboard, sobre la que se apoyan varias de las demás. Calcula el valor neto de las ventas, después de aplicar los descuentos.

```
VentasTotales =  
SUMX(  
    DetalleFacturas,  
    DetalleFacturas[Subtotal] * (1 - DetalleFacturas[Descuento] / 100)  
)
```

Esta medida requiere una explicación detenida, porque su primera versión estaba equivocada y el error solo se hizo visible al probar el dashboard.

La versión inicial y su problema. La primera formulación sumaba directamente la columna de total de la tabla de Facturas. Funcionaba correctamente para el total global y para los filtros de ciudad o método de pago, que operan a nivel de factura. El problema apareció al filtrar por marca de producto: los resultados quedaban inflados, y una medida derivada llegaba a dar valores negativos.

La causa, un problema de granularidad. La granularidad se refiere al nivel de detalle al que vive cada dato. El total de la factura vive a nivel de factura, mientras que la marca del producto vive a nivel de línea de detalle. Son niveles distintos. Cuando se filtraba por Lenovo, Power BI tomaba todas las facturas que contuvieran algún producto Lenovo y sumaba su total completo, incluyendo el valor de los productos de otras marcas que iban en esa misma factura. El resultado no era la venta de Lenovo, sino la venta de todas las facturas donde apareció Lenovo.

La solución. La función SUMX recorre una tabla fila por fila, evalúa una expresión en cada una y suma los resultados. Al aplicarla sobre la tabla de detalle, la medida calcula la venta neta de cada línea individual, y cada línea sabe a qué producto y marca pertenece. De ese modo, al filtrar por Lenovo, solo se recorren y suman las líneas de Lenovo. La medida quedó correcta para cualquier filtro.

Se verificó que ambas versiones produjeran el mismo total global, lo que confirma que la corrección no alteró las cifras generales sino que ajustó su comportamiento ante los filtros.

CantidadFacturas

Cuenta las ventas realizadas. Sirve de apoyo para el cálculo del ticket promedio.

```
CantidadFacturas = COUNTROWS(Facturas)
```

TicketPromedio

Es el valor promedio de una venta, y responde a la pregunta de cuánto gasta en promedio un cliente en cada compra.

```
TicketPromedio =  
DIVIDE(  
    [VentasTotales],  
    [CantidadFacturas],  
    0  
)
```

Se emplea la función `DIVIDE` en lugar del operador de división convencional porque `DIVIDE` gestiona el caso de la división por cero. Si un filtro dejara el conjunto sin facturas, la división normal produciría un error visible en pantalla, mientras que `DIVIDE` devuelve el tercer argumento, que en este caso es cero. Es una práctica defensiva estándar en DAX.

ProductosVendidos

Cuenta el total de unidades vendidas, sumando las cantidades de todas las líneas de detalle.

```
ProductosVendidos = SUM(DetalleFacturas[Cantidad])
```

Vale la pena notar que esta medida suma una columna que vive en una tabla distinta a la que la contiene, y funciona gracias a las relaciones del modelo. Power BI sabe cómo conectar Facturas con DetalleFacturas, y por eso puede resolver el cálculo. Es una demostración práctica de por qué el modelo relacional del Avance 2 importaba.

VentaBruta

Representa el valor que se habría facturado a precio de lista, sin aplicar ningún descuento. Sirve de base para el cálculo del margen.

```
VentaBruta = SUM(DetalleFacturas[Subtotal])
```

MargenVentas

Mide qué proporción del valor de lista conserva la empresa después de aplicar los descuentos.

```
MargenVentas =  
DIVIDE(  
    [VentasTotales],  
    [VentaBruta],  
    0  
)
```

Una decisión metodológica sobre esta medida. El margen, en su definición contable estricta, es la diferencia entre el ingreso y el costo, expresada como porcentaje. El modelo de datos de TechCore no contiene el costo de adquisición de los productos, solo su precio de venta. Sin el costo, no es posible calcular un margen de rentabilidad real. Ante esa limitación había dos caminos: suponer un costo estimado, o redefinir la métrica sobre los datos disponibles.

Se optó por lo segundo. El margen se define aquí como la relación entre la venta neta y la venta bruta, midiendo el efecto de la política de descuentos sobre el ingreso. Un margen del noventa y ocho coma cuarenta y cinco por ciento significa que la empresa conserva ese porcentaje de su precio de lista, cediendo el resto en descuentos. La razón de esta elección es que se trata de una métrica calculable con datos reales y verificables, mientras que un margen apoyado en un costo supuesto habría producido un número arbitrario, sin sustento en la información de la empresa.

DescuentoOtorgado

Expresa en valor absoluto cuánto ingreso cedió la empresa por su política de descuentos.

$$\text{DescuentoOtorgado} = [\text{VentaBruta}] - [\text{VentasTotales}]$$

%ParticipacionCiudad

Indica qué porcentaje del total de ventas de la cadena aporta cada ciudad. Es la medida más interesante desde el punto de vista técnico.

$$\%ParticipacionCiudad =$$

```

DIVIDE(
    [VentasTotales],
    CALCULATE(
        [VentasTotales],
        REMOVEFILTERS(Ciudades)
    ),
    0
)

```

La clave está en la función REMOVEFILTERS. Para entender por qué es necesaria, conviene pensar en cómo se evalúa la medida dentro de una tabla que muestra las ciudades. Cuando Power BI calcula la fila de Medellín, el contexto activo es Medellín, de modo que VentasTotales devuelve las ventas de esa ciudad. Si el denominador usara la misma expresión, también daría las ventas de Medellín, y la división produciría siempre cien por ciento.

La función CALCULATE permite modificar el contexto de un cálculo, y REMOVEFILTERS es la instrucción que retira el filtro de ciudad. Así, el denominador calcula las ventas de todas las ciudades, mientras el numerador conserva el filtro de la ciudad actual. La división entre ambos entrega la participación real.

%ParticipacionMarca

Aplica la misma lógica a las marcas de producto, retirando el filtro de la tabla de Productos.

```

%ParticipacionMarca =
DIVIDE(
    [VentasTotales],

```

```

CALCULATE(
    [VentasTotales],
    REMOVEFILTERS(Productos)
),
0
)

```

Las Jerarquías de Navegación

Una jerarquía permite navegar dentro de una misma visualización desde una vista general hacia el detalle. En un gráfico con jerarquía de fecha, el usuario ve inicialmente los años, y mediante los controles de profundización puede descender a los meses de un año concreto, y de ahí a los días de ese mes. Es lo que distingue un gráfico estático de uno explorable.

Jerarquía de Fecha

Se construyó sobre la tabla de Calendario, con tres niveles: año, nombre del mes y día. Se aplicó al gráfico de evolución de las ventas, que muestra los años de forma predeterminada y permite descender al detalle mensual y diario.

Jerarquía de Ubicación

Esta jerarquía presentó un obstáculo técnico que vale la pena explicar. Una jerarquía debe construirse dentro de una sola tabla, pero el nombre de la ciudad reside en la tabla de Ciudades y el de la sucursal en la tabla de Sucursales. Los dos campos no comparten tabla, así que no era posible armarla de forma directa.

La solución consistió en llevar el nombre de la ciudad a la tabla de Sucursales mediante una columna calculada.

```
NombreCiudad = RELATED(Ciudades[NombreCiudad])
```

La función RELATED recupera un valor desde una tabla relacionada. Como Sucursales está relacionada con Ciudades a través de la llave foránea, RELATED puede traer el nombre de la ciudad correspondiente a cada sucursal. Con ambos campos en la misma tabla, la jerarquía quedó construida con la ciudad como primer nivel y la sucursal como segundo. Se aplicó al gráfico de ventas por sucursal, que muestra las ciudades y permite descender a las tiendas de cada una.

Las Segmentaciones Dinámicas

Los segmentadores son los controles que permiten al usuario del dashboard filtrar la información. Se implementaron los cinco que solicita la consigna.

Segmentador	Campo	Tipo de control
Ciudad	Ciudades, NombreCiudad	Lista de selección
Marca de producto	Productos, Marca	Lista de selección única
Método de pago	Facturas, MetodoPago	Lista desplegable
Período de análisis	Calendario, Date	Control deslizante de rango
Rango de edad	Clientes, Edad	Control deslizante numérico

El segmentador de marca se configuró en modo de selección única, por una razón que se explica en la sección siguiente: la funcionalidad del eje dinámico requiere saber si el usuario ha seleccionado exactamente una marca.

Las Visualizaciones

El dashboard incorpora las visualizaciones solicitadas por la consigna, más algunas complementarias.

Tarjetas de indicadores clave. Cuatro tarjetas en la franja superior muestran los valores globales del negocio: ingresos por ventas, unidades vendidas, ticket promedio y margen de ventas. Se ubican arriba porque son las cifras que el lector busca primero.

Mapa geográfico. Ubica las cuatro ciudades sobre el territorio colombiano, con el tamaño de cada burbuja proporcional a sus ventas. Comunica de un vistazo la concentración geográfica del negocio, con Medellín y Bogotá dominando visualmente el mapa.

Gráfico de barras por marca. Muestra las ventas de cada marca, ordenadas de mayor a menor. Incorpora una funcionalidad avanzada que se explica más adelante.

Línea de evolución de las ventas. Presenta el comportamiento de las ventas a lo largo del tiempo, con la jerarquía de fecha aplicada. Un detalle importante de esta visualización es que el eje vertical arranca en cero, lo que evita exagerar visualmente las variaciones. Un eje truncado habría hecho parecer dramáticas unas diferencias que en realidad son moderadas.

Gráfico de ventas por sucursal. Aplica la jerarquía de ubicación, mostrando las ciudades y permitiendo descender a sus sucursales. Reemplazó a un gráfico circular inicial, porque las porciones de tamaño similar resultan difíciles de comparar visualmente, mientras que las barras permiten una lectura inmediata.

Tarjetas de contexto. Dos tarjetas señalan la ciudad con más ventas y el producto más vendido, aportando el dato concreto sin obligar al lector a interpretarlo desde un gráfico.

Funcionalidad Avanzada: el Eje Dinámico

El gráfico de ventas por marca incorpora una funcionalidad que va más allá de lo solicitado: cambia su nivel de análisis según la selección del usuario. Cuando no hay ninguna marca seleccionada, muestra las ventas de todas las marcas. Cuando el usuario selecciona una marca en el segmentador, el mismo gráfico deja de mostrar marcas y pasa a mostrar los productos de esa marca. Al retirar la selección, regresa a la vista general.

Todo esto ocurre en un único gráfico, sin duplicar visualizaciones ni recurrir a botones adicionales.

El problema que resuelve

Un gráfico convencional con la marca en el eje se comporta de forma limitada al filtrar: si el usuario selecciona Lenovo, el gráfico simplemente muestra una sola barra con Lenovo, porque su eje sigue siendo el campo de marca. La información no se enriquece, solo se reduce. Para ver el detalle de los productos de esa marca haría falta otro gráfico distinto.

La solución técnica

Se construyó una tabla auxiliar, sin relaciones con el resto del modelo, que reúne en una sola columna tanto los nombres de las marcas como los de los productos.

```
Eje Dinamico =  
VAR ListaElementos =  
    UNION(
```

```

        SELECTCOLUMNS(Productos, "Elemento", Productos[Marca]),
        SELECTCOLUMNS(Productos, "Elemento", Productos[NombreProducto])
    )
RETURN
    FILTER(
        DISTINCT(ListaElementos),
        NOT ISBLANK([Elemento])
    )

```

La función `SELECTCOLUMNS` extrae una columna y le asigna un nombre común, en este caso `Elemento`. Se aplica dos veces, una sobre las marcas y otra sobre los nombres de producto. La función `UNION` combina ambas listas en una sola tabla, `DISTINCT` elimina los valores repetidos, y el filtro descarta los vacíos. El resultado es una columna que contiene, mezclados, todos los nombres de marca y todos los nombres de producto.

Esta tabla se mantuvo deliberadamente desconectada del modelo. La razón es que contiene simultáneamente dos tipos de entidad, marcas y productos, de modo que una relación física no tendría sentido. En su lugar, el filtrado se aplica desde una medida, de forma controlada.

La medida que decide el nivel

La medida que alimenta el gráfico evalúa, en cada barra, si debe interpretar el elemento como una marca o como un producto.

```

Ventas Eje Dinamico =
VAR ElementoActual =
    SELECTEDVALUE('Eje Dinamico'[Elemento])

```

```

VAR HayUnaMarcaSeleccionada =
    HASONEVALUE(Productos[Marca])
RETURN
    IF(
        HayUnaMarcaSeleccionada,
        CALCULATE([VentasTotales],
            TREATAS({ElementoActual}, Productos[NombreProducto])),
        CALCULATE([VentasTotales],
            TREATAS({ElementoActual}, Productos[Marca]))
    )

```

La función `SELECTEDVALUE` identifica qué elemento está evaluando el gráfico en cada barra. La función `HASONEVALUE` detecta si el usuario ha seleccionado exactamente una marca en el segmentador. Con esa información, la medida decide el camino.

El elemento central de la solución es la función `TREATAS`, que permite tomar el valor de la tabla desconectada y aplicarlo como filtro sobre una columna del modelo. Cuando no hay marca seleccionada, el elemento se trata como una marca y se filtra sobre la columna de marca. Cuando hay una marca seleccionada, el mismo elemento se trata como un nombre de producto y se filtra sobre la columna de producto. La misma columna del eje cumple así dos funciones distintas según el contexto.

Los elementos que no corresponden al nivel activo devuelven un valor vacío, y Power BI los oculta automáticamente al mantener desactivada la opción de mostrar elementos sin datos. Por eso, sin selección solo aparecen las marcas, y con Lenovo seleccionado solo aparecen los productos de Lenovo.

El valor de esta funcionalidad

La navegación quedó organizada en dos niveles dentro de un solo espacio visual: el nivel de marca y el nivel de producto de la marca seleccionada. El usuario puede pasar de una vista consolidada a una detallada sin cambiar de página ni consultar otro gráfico. Es una forma de aprovechar el segmentador como control de navegación, más allá de su función de filtro.

Decisiones de Diseño

El diseño del panel siguió una lógica de lectura, no una estética arbitraria. El ojo recorre la pantalla de arriba abajo y de izquierda a derecha, y la disposición de los elementos aprovecha ese recorrido natural.

La franja superior. Contiene el título y los cuatro indicadores globales, que responden a la pregunta inicial de cualquier lector: cómo va el negocio. Son las cifras que se leen primero.

La columna izquierda. Agrupa los segmentadores, disponibles para el usuario pero sin competir con los datos por la atención. Se optó por controles compactos, en formato desplegable o deslizable, para que los cinco filtros ocupen poco espacio y no obliguen a desplazarse.

El cuerpo central. Distribuye las visualizaciones según la pregunta que responden. Dónde se vende, en el mapa. Qué se vende, en el gráfico de marcas. Cómo evoluciona, en la línea de tiempo. Y el detalle por sucursal, que respalda la lectura geográfica.

La precisión de las cifras. Las tarjetas se configuraron con dos decimales. En su formato inicial, sin decimales, Power BI redondeaba las ventas de dos coma cincuenta y tres billones a tres billones, una diferencia de casi medio billón de pesos. Para un directivo que toma decisiones con esa cifra, la imprecisión no es aceptable.

Resumen del Avance

El Avance 3 convirtió el modelo relacional en una herramienta de análisis. Se importaron las siete tablas a Power BI, se verificaron las seis relaciones y se construyó una tabla de calendario dedicada. Se crearon nueve medidas DAX que calculan los indicadores clave del negocio, incluidas dos medidas de participación que emplean la manipulación del contexto de filtro mediante `CALCULATE` y `REMOVEFILTERS`.

Se definieron dos jerarquías de navegación, la de fecha y la de ubicación, esta última resuelta mediante una columna calculada con `RELATED` para reunir en una misma tabla dos campos que residían en tablas distintas. Se implementaron cinco segmentaciones dinámicas y las visualizaciones solicitadas, más una funcionalidad adicional de eje dinámico que permite navegar de las marcas a sus productos dentro de un mismo gráfico.

El proceso incluyó la detección y corrección de un error de granularidad en la medida principal, que solo se hizo visible al probar el comportamiento del dashboard ante el filtro de marca. Ese hallazgo confirma que un dashboard debe probarse en uso, no solo construirse. El panel resultante permite a la dirección de TechCore explorar su desempeño comercial de forma interactiva, y las conclusiones derivadas de ese análisis se presentan en el documento de conclusiones y recomendaciones.